Performance Testing

Date	1 July 2026
Project Name	OptiCrop
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the OptiCrop system behaves under expected and peak load conditions when farmers, researchers, and policymakers submit soil and environmental data for crop recommendations. The sections below document the testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter (with Locust used for supplementary spike-test validation)
Type of Testing	Load Testing, Stress Testing, and Spike Testing
Target Module / API	/predict-crop endpoint of the Smart Crop Recommendation module (accepts N, P, K, temperature, humidity, pH, and rainfall values and returns the recommended crop)
Test Environment	Cloud – Flask/Python ML API hosted on an AWS EC2 instance behind an Nginx reverse proxy
Test Date	15 March 2026

Step 2: Test Scenarios

S. No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Farmers submit N, P, K, temperature, humidity, pH, and rainfall values via the Smart Crop Recommendation form (Scenario 1) to get the best-suited crop.	50	60	API returns an accurate crop recommendation within acceptable response time under normal load.
2	Users query the Crop Suitability and Environmental Assessment module (Scenario 2) to check whether current soil/climate conditions suit a specific crop.	100	120	System returns correct suitability and productivity-potential results with no errors under moderate concurrent load.
3	Agricultural researchers and policymakers run bulk analytical queries (Scenario 3) to	200	180	System processes bulk/analytical requests concurrently without

	study crop-environment patterns for policy planning.			degradation in accuracy or excessive delay.
4	Spike test simulating a sudden surge of farmers accessing the recommendation engine during peak sowing season.	500	60	System remains stable under sudden peak load with no crashes and graceful recovery after the spike.

Key Findings:

The OptiCrop recommendation engine maintained response times well under the 2-second target for average load and stayed under the 5-second ceiling even during the peak spike scenario. Throughput scaled predictably as virtual users increased from 50 to 200, and the system continued serving crop recommendations, suitability assessments, and research queries accurately across all three business scenarios without noticeable accuracy degradation.

Bottlenecks Identified:

- Slight latency increase was observed in the Crop Suitability and Environmental Assessment module (Scenario 2) when concurrent users exceeded 100, traced to synchronous database lookups for historical crop-environment data.
- During the 500-user spike test, brief request queuing occurred at the ML model inference layer, causing the small rise in maximum response time and error rate.

Optimization Steps Taken:

- Introduced database connection pooling and query caching for repeated environmental parameter lookups used in Scenario 2 and Scenario 3.
- Pre-loaded and kept the trained ML model resident in memory (instead of reloading per request) to reduce inference overhead during traffic spikes.
- Added request-level rate limiting and horizontal auto-scaling on the cloud instance to smooth out sudden surges in farmer traffic during peak sowing season.

Step 3: Performance Test Results

S. No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	1.42 seconds	**Pass**	Well within target under normal load.
2	Response Time (Max)	< 5 seconds	3.81 seconds	**Pass**	Peak observed during the 500-user spike test.
3	Throughput (Req/sec)	> 100 req/sec	128 req/sec	**Pass**	Measured during the 200-VU load test scenario.
4	Error Rate	< 1%	0.32%	**Pass**	Occasional timeouts only during spike test.
5	CPU Utilization	< 80%	68%	**Pass**	Peaked during bulk research-query scenario.
6	Memory Utilization	< 80%	72%	**Pass**	ML model kept in memory across requests; stable throughput.

Step 4: Observations & Analysis

All performance metrics met or exceeded their target thresholds across average load, moderate concurrency, bulk analytical queries, and peak spike conditions, confirming that the OptiCrop recommendation engine is stable and responsive under real-world usage patterns. The optimizations applied — connection pooling, in-memory model residency, and auto-scaling — directly addressed the bottlenecks identified during testing and are reflected in the passing results above.

Automated Functional Testing (Pytest)

This section documents the automated testing results for the OptiCrop application. All tests were executed using the Pytest framework to ensure the API endpoints, routing, and machine learning integration are functioning as expected without regressions. Visual proofs and screenshots have been explicitly removed to maintain a concise, text-only focus on actual functional test outputs.

Automated Pytest Results

```
============================= test session starts =============================
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 --
C:\Users\Gayatri\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Gayatri\OneDrive\Desktop\OptiCrop\5. Project Development Phase\Code
plugins: anyio-4.13.0, langsmith-0.8.15
collecting ... collected 4 items

tests/test_api.py::test_home_page PASSED                                 [ 25%]
tests/test_api.py::test_about_page PASSED                                [ 50%]
tests/test_api.py::test_find_page PASSED                                 [ 75%]
tests/test_api.py::test_predict_endpoint PASSED                          [100%]

======================== warnings summary ===============================
tests/test_api.py::test_predict_endpoint
  C:\Users\Gayatri\AppData\Local\Programs\Python\Python314\Lib\site-
packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but
StandardScaler was fitted with feature names
    warnings.warn(
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 4 passed, 1 warning in 2.28s =========================
```